Report

Snumber: s1122062

Name: Levente Bódi

1. Summary of the article by Kamp

In “You’re Doing It Wrong,” Poul-Henning Kamp argues that much of computer science remains disconnected from real-world computing because it still relies on outdated models of performance that ignore modern hardware realities. Through illustration from his own experience with high-performance HTTP accelerator Varnish, Kamp shows that even "optimal" algorithms, such as the binary heap, can be ten times slower when implemented in practice, taking into account virtual memory and caching behavior. He argues that algorithms have been analyzed for years using Big-O analysis that started in the 1960s, noting that a single page fault on modern systems can discard millions of instructions. The issue that he describes is not the paging itself, but how the field of computer science continues to be oriented to educating and creating algorithms in theoretical abstraction to the nature of hardware, creating inefficiencies in performance, hardware utilization, and energy consumption. Kamp appeals to a shift to algorithmic thought in regard to the actual computation setting, closing the gaps between theory and practice.

2. An explanation why the first generation in a B-heap does not expand, and an explanation of whether this works for all page sizes

In a B-heap, the first generation of items in a new page does not increase such that both children of a parent reside on the same virtual-memory page. This minimizes costly page crossings for comparisons and swaps, increasing spatial locality and reduced page faults. The arrangement also reflects the fact that real systems favor memory access patterns that are not split across a page or cache line but rather across them. Because page sizes are two's powers (e.g., 4096 bytes), this structure maps well to memory addressing and gives uniform mapping in virtual memory. Though it partially limits tree expansion, it improves performance considerably by making optimal use of the memory hierarchy's actual behavior.

3. Baseline measurements and a description of the machine on which you test

Baseline measurements when running main.cpp at first:

user time: 32.092734 s

soft page faults: 524398

hard page faults: 0

max memory: 2099712 KiB

voluntary context switches: 4

involuntary context switches: 79

dummy value (ignore): 549755813888

typical page size: 4096

const int64_t SIZE = 16384ULL;

const int64_t REPEAT = 1ULL;

The machine:

Linux DESKTOP-O3TESH8 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 5 18:30:46 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux

No LSB modules are available.

Distributor ID: Ubuntu

Description: Ubuntu 20.04.6 LTS

Release: 20.04

Codename: focal

Architecture: x86_64

CPU op-mode(s): 32-bit, 64-bit

Byte Order: Little Endian

Address sizes: 39 bits physical, 48 bits virtual

CPU(s): 12

On-line CPU(s) list: 0-11

Thread(s) per core: 2

Core(s) per socket: 6

Socket(s): 1

NUMA node(s): 1

Vendor ID: GenuineIntel

CPU family: 6

Model: 141

Model name: 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz

Stepping: 1

CPU MHz: 2688.000

BogoMIPS: 5376.00

Virtualization: VT-x

Hypervisor vendor: Microsoft

Virtualization type: full

L1d cache: 288 KiB

L1i cache: 192 KiB

L2 cache: 7.5 MiB

L3 cache: 12 MiB

H/W path Device Class Description

==========

system Computer

/0 bus Motherboard

/0/0 memory 3968MiB System memory

/0/1 processor 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz

/0/2 display Microsoft Corporation

/0/3 generic Red Hat, Inc.

/0/3/0 generic Virtual I/O device

/0/4 storage Virtio console

/0/4/0 generic Virtual I/O device

/0/5 display Microsoft Corporation

/0/6 system PnP device PNP0b00

/0/7 scsi0 storage

/0/7/0.0.0 /dev/sda volume 388MiB Virtual Disk

/0/7/0.0.1 /dev/sdb volume 186MiB Virtual Disk

/0/7/0.0.2 /dev/sdc volume 1GiB Virtual Disk

/0/7/0.0.3 /dev/sdd volume 1TiB Virtual Disk

/1 eth0 network Ethernet interface

4. A list of all the changes you made and for every change, why you think it will improve the performance. Support your claims with measurements

First, as described by the book, I changed the order to row-major of the loop by filling the image with zeroes, and also by the filtering. What I got is impressive:

user time: 14.327895 s

soft page faults: 524397

hard page faults: 0

max memory: 2099968 KiB

voluntary context switches: 1

involuntary context switches: 24

dummy value (ignore): 549755813888

typical page size: 4096

This already reduced a lot of time and context switches. After this, I made a small modification on the most-inner i loop, namely, I make a constant of 1/9 instead of dividing by 9 at each loop, and use this precomputed constant in the loop. Besides that, I take row pointers to avoid recomputing base index by all loops on j, this enables vectorization. In the original code, each access required computing (row*SIZE + col), now, the expensive row*SIZE part is done once per row per iteration, not 9× per pixel. In the most-inner i loop I also replaced the filter by an explicit sum of the 9 neighbours. Before, the CPU executed two nested loops, loop counters, compares, branches, and index recomputation each time, now, there are no inner loop counters, no branch mispredictions, no induction-variable updates, only straight line-adds.

After implementing these I got:

user time: 11.679288 s

soft page faults: 524397

hard page faults: 0

max memory: 2099840 KiB

voluntary context switches: 0

involuntary context switches: 19

dummy value (ignore): 549755813888

typical page size: 4096

It had a significant time improvement, and an improvement regarding context switches.

I also adjusted the dummy variable accumulation to row-major, just as by the codeblocks above.

After this I got:

user time: 6.273525 s

soft page faults: 524396

hard page faults: 0

max memory: 2099968 KiB

voluntary context switches: 0

involuntary context switches: 9

dummy value (ignore): 549755813888

typical page size: 4096

It got a bit better in user time, but not significantly, however, a small improvement is also an improvement.

I ran it again, and it was impressive for the 2nd try:

user time: 2.044070 s

soft page faults: 524397

hard page faults: 0

max memory: 2099968 KiB

voluntary context switches: 0

involuntary context switches: 7

dummy value (ignore): 549755813888

typical page size: 4096

Going from approximately 32 seconds to two seconds is a 16times better optimization, which seems impressive. Running it again gave approximately the same result:

user time: 2.198849 s

soft page faults: 524399

hard page faults: 0

max memory: 2099968 KiB

voluntary context switches: 1

involuntary context switches: 11

dummy value (ignore): 549755813888

typical page size: 4096

5. The final measurements and a comparison with the baseline measurements and explain any remaining page faults from the theory in the book

For the final measurements I also added wall time and system time.

wall time: 3.330924 s

user time: 1.959894 s

system time: 1.371925 s

soft page faults: 524397

hard page faults: 1

max memory: 2099840 KiB

voluntary context switches: 1

involuntary context switches: 7

dummy value (ignore): 549755813888

typical page size: 4096

This was the initial:

user time: 32.092734 s

soft page faults: 524398

hard page faults: 0

max memory: 2099712 KiB

voluntary context switches: 4

involuntary context switches: 79

dummy value (ignore): 549755813888

typical page size: 4096

So, the user time improved almost by 16times, or 1600%, there were no hard page faults, and the involuntary and voluntary context switches dropped from 79 and 4 to 7 and 1. This seems to be a significant improvement. There are no hard page faults remaining. Other than that, soft page faults occur when memory pages are already in RAM but not yet mapped to the process, so they do not require slow disk access. They mainly result from first-time page allocation and do not affect performance, so they can be ignored in the analysis.